TP DevSecOps – Pipeline CI/CD Sécurisé & Kubernetes

- 1 TP DevSecOps – Pipeline CI/CD Sécurisé & Kubernetes
  - 1.1 Légende
  - 1.2 Table des matières
  - 1.3 Présentation du TP
  - 1.4 Partie 1 – Mise en place de l'environnement
  - 1.5 Partie 2 – Conteneurisation sécurisée
  - 1.6 Partie 3 – Pipeline CI/CD avec GitHub Actions
  - 1.7 Partie 4 – Manifestes Kubernetes
  - 1.8 Partie 5 – Sécurité avancée avec Trivy Operator
  - 1.9 Partie 6 – Questions de synthèse
  - 1.10 Livrables et évaluation
  - 1.11 Ressources

1 TP DevSecOps – Pipeline CI/CD Sécurisé & Kubernetes

5ème Année – Spécialité Cloud & Réseau | Durée : 6 heures ESGI – Département Cloud & Réseau | 2026

1.1 Légende

📋 **Tâche pratique** — action à réaliser, résultat à capturer dans votre rendu (screenshot, sortie de commande, fichier)

💬 **Question de réflexion** — réponse écrite attendue dans votre document de rendu

---

1.2 Table des matières

1. Présentation du TP
2. Partie 1 – Mise en place de l'environnement
3. Partie 2 – Conteneurisation sécurisée
4. Partie 3 – Pipeline CI/CD avec GitHub Actions
5. Partie 4 – Manifestes Kubernetes
6. Partie 5 – Sécurité avancée avec Trivy Operator
7. Partie 6 – Questions de synthèse
8. Livrables et évaluation
9. Ressources

1.3 Présentation du TP

1.3.1 Contexte

Vous êtes recruté·e comme **DevSecOps Engineer** dans une fintech en pleine croissance. L'équipe dispose d'une application web (un gestionnaire de tâches en Python/Flask) mais **aucune pratique DevSecOps n'est en place** : les images Docker ne sont pas scannées, le déploiement est entièrement manuel, et les manifestes Kubernetes ne respectent pas les bonnes pratiques de sécurité.

Votre mission est de construire de zéro un pipeline CI/CD sécurisé qui :

1. **Vérifie** la qualité du Dockerfile avec un linter
2. **Scanne** l'image Docker avec **Trivy** et bloque le déploiement en cas de vulnérabilité `HIGH` ou `CRITICAL`
3. **Publie** l'image validée sur un registre de conteneurs
4. **Déploie** automatiquement sur Kubernetes après validation de la sécurité

> Ce TP est intentionnellement moins guidé que le précédent. Vous devez faire preuve d'initiative, consulter la documentation officielle et prendre des décisions d'architecture justifiées.

1.3.2 Ce qui vous est fourni

Élément	Description
`app/app.py`	Code source de l'application Flask — **ne pas modifier**
`app/requirements.txt`	Dépendances Python avec des versions intentionnellement anciennes
`app/templates/index.html`	Interface web — **ne pas modifier**

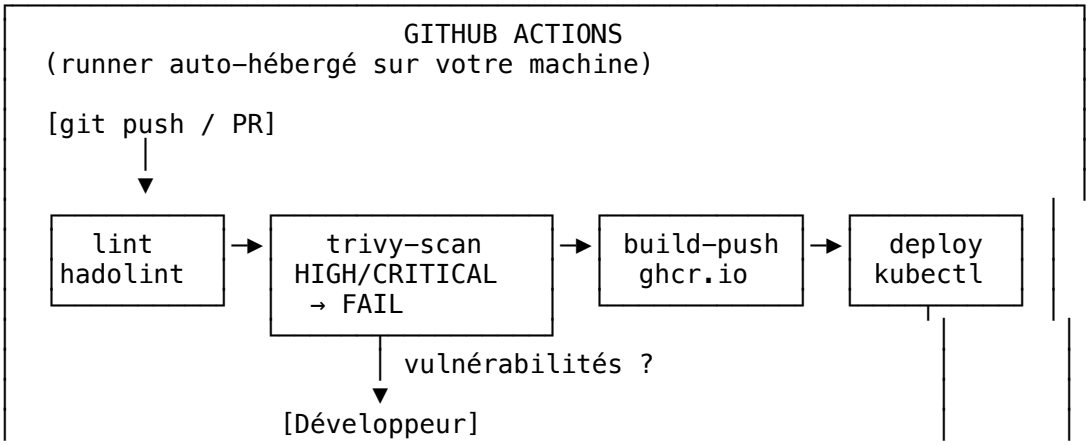
> Le `requirements.txt` contient des versions volontairement obsolètes présentant des CVEs connues. Votre pipeline devra les détecter et vous devrez les corriger pour que le déploiement puisse avoir lieu.

1.3.3 Ce que vous devez produire entièrement

- Un `Dockerfile` sécurisé
- Un pipeline GitHub Actions `.github/workflows/ci-cd.yml`
- Tous les manifestes Kubernetes dans un dossier `k8s/`
- Les réponses aux questions et captures des tâches pratiques

Aucun YAML Kubernetes ni fichier de pipeline ne vous est fourni.

1.3.4 Architecture cible



```
│                  corrige les CVEs                                │        │
└──────────────────────────────────────────────────────────────────┘        │
                                                                            ▼
                                                   ┌──────────────────────────────┐
                                                   │ CLUSTER KUBERNETES           │
                                                   │ (minikube)                   │
                                                   │                              │
                                                   │ namespace: taskmanager       │
                                                   │                              │
                                                   │  ┌────────────────────┐      │
                                                   │  │ Deployment (×2)    │      │
                                                   │  │   taskmanager      │      │
                                                   │  └────────────────────┘      │
                                                   │           │                  │
                                                   │           ▼                  │
                                                   │  ┌────────────────────┐      │
                                                   │  │      Redis         │      │
                                                   │  └────────────────────┘      │
                                                   │                              │
                                                   │ + Service NodePort           │
                                                   │ + Ingress                    │
                                                   │ + NetworkPolicy              │
                                                   │ + RBAC                       │
                                                   │ + HPA                        │
                                                   └──────────────────────────────┘
```

1.3.5 Prérequis

Vérifiez que chaque outil est installé et fonctionnel **avant de commencer**. Si un outil manque, installez-le.

Outil	Rôle	Documentation
minikube ≥ 1.32	Cluster Kubernetes local	https://minikube.sigs.k8s.io
kubectl ≥ 1.28	CLI Kubernetes	https://kubernetes.io/docs/tasks/tools/
Docker ≥ 24	Build et exécution d'images	https://docs.docker.com
Trivy ≥ 0.50	Scanner de vulnérabilités	https://trivy.dev
Helm ≥ 3.12	Gestionnaire de paquets Kubernetes	https://helm.sh/docs/intro/install/
Git	Versioning	—
Compte GitHub	CI/CD + registre d'images	https://github.com

1.4 Partie 1 – Mise en place de l'environnement

Durée estimée : 30 minutes

1.4.1 Objectifs

- Initialiser le dépôt GitHub avec le code source fourni
- Configurer un runner GitHub Actions auto-hébergé
- Préparer et vérifier le cluster Kubernetes

1.4.2 1.1 Dépôt GitHub

Créez un **nouveau dépôt GitHub public** nommé `esgi-devsecops-tp`. Poussez-y le code source fourni. La structure finale attendue dans votre dépôt est la suivante :

```
esgi-devsecops-tp/
├── app/
│   ├── app.py
│   ├── requirements.txt
│   └── templates/
│       └── index.html
├── k8s/                        ← à créer (vos manifestes)
│   └── ...
├── .github/
│   └── workflows/
│       └── ci-cd.yml           ← votre pipeline
└── Dockerfile                  ← votre Dockerfile
```

📋 **T1.1** — Incluez dans votre rendu l'URL de votre dépôt GitHub.

1.4.3 1.2 Runner GitHub Actions auto-hébergé

Le job `deploy` de votre pipeline doit accéder à votre cluster minikube local. Les runners cloud de GitHub (`ubuntu-latest`) n'ont pas accès à votre réseau local : vous devez configurer un **runner auto-hébergé** sur votre propre machine.

À faire :

- Dans votre dépôt GitHub : **Settings → Actions → Runners → New self-hosted runner**
- Suivez les instructions affichées par GitHub pour installer et démarrer le runner
- Vérifiez que le runner apparaît avec le statut **Idle** dans l'interface GitHub avant de continuer

📋 **T1.2** — Capturez un screenshot de l'interface GitHub montrant votre runner en statut **Idle**.

💬 **Q1.1** — Quelle est la différence principale entre un runner GitHub-hosted et un runner auto-hébergé ? Dans quels cas une entreprise préférerait-elle un runner auto-hébergé ?

1.4.4 1.3 Cluster Kubernetes

Démarrez votre cluster minikube avec les ressources suffisantes pour ce TP. Le cluster doit disposer des addons `ingress` et `metrics-server`.

📋 **T1.3** — Après démarrage du cluster, copiez dans votre rendu la sortie des commandes suivantes :

```
kubectl get nodes
kubectl get pods -n kube-system
minikube addons list | grep -E "enabled|ingress|metrics"
```

💬 **Q1.2** — Listez les addons que vous avez activés et expliquez l'utilité de chacun dans le contexte de ce TP.

1.5 Partie 2 – Conteneurisation sécurisée

Durée estimée : 45 minutes

1.5.1 Objectifs

- Écrire un Dockerfile sécurisé respectant les bonnes pratiques
- Utiliser un build multi-stage pour minimiser la surface d'attaque
- Tester l'image localement et effectuer un premier scan Trivy

1.5.2 2.1 Contraintes du Dockerfile

Vous devez écrire un `Dockerfile` à la racine du dépôt qui respecte **impérativement** les contraintes suivantes :

Contrainte	Exigence
Build multi-stage	Obligatoire — séparer l'étape de build de l'image finale
Image de base runtime	Image légère : `alpine`, `slim` ou `distroless`
Utilisateur	L'application **ne doit pas** tourner en `root`
Port exposé	Port `5000` uniquement
Commande de démarrage	Utiliser `gunicorn` (pas `flask run`)
Secrets / credentials	Aucune valeur sensible en dur dans le Dockerfile

L'application Flask se démarre avec gunicorn de la façon suivante :

```
gunicorn --bind 0.0.0.0:5000 --workers 2 app:app
```

> **Note technique** : `gunicorn 20.x` requiert le module `pkg_resources` (fourni par `setuptools`). Si votre image Alpine ne démarre pas avec une erreur `ModuleNotFoundError: No module named 'pkg_resources'`, assurez-vous que `setuptools` est installé dans l'image finale, ou utilisez une image base `slim`.

> 📋 **T2.1** — Construisez votre image localement et copiez la commande utilisée ainsi que la taille finale de l'image (sortie de `docker images taskmanager`). Comparez cette taille avec ce qu'elle serait sans multi-stage (vous pouvez l'estimer ou le tester).

> 💬 **Q2.1** — Expliquez l'intérêt du build multi-stage pour la sécurité des images Docker. Pourquoi le stage de build ne doit-il pas se retrouver dans l'image finale ?

> 💬 **Q2.2** — Pourquoi est-il problématique de faire tourner un conteneur en tant que `root` ? Citez deux risques de sécurité concrets.

1.5.3 2.2 Vérification locale du fonctionnement

Avant d'intégrer votre Dockerfile dans le pipeline, vérifiez son bon fonctionnement en local.

> 📋 **T2.2** — Démarrez votre image sans Redis et copiez dans votre rendu le résultat des appels suivants :
>
> ```
> curl http://localhost:5000/healthz
> curl http://localhost:5000/readyz
> ```
>
> Expliquez pourquoi `/readyz` retourne un code d'erreur dans ce contexte.

📋 **T2.3** — Vérifiez que l'application ne tourne pas en root à l'intérieur du conteneur. Copiez la sortie de la commande permettant de le confirmer (indice : `docker exec`).

1.5.4 2.3 Découverte de Trivy en local

📋 **T2.4** — Scannez votre image avec Trivy en filtrant sur les sévérités `HIGH` et `CRITICAL`. Copiez **l'intégralité de la sortie du scan** dans votre rendu.

📋 **T2.5** — À partir des résultats du scan, remplissez le tableau suivant dans votre rendu :

CVE ID	Sévérité	Package	Version installée	Version corrigée
…	…	…	…	…

Mettez ensuite à jour `requirements.txt` avec les versions corrigées et commitez.

💬 **Q2.3** — Pour chaque CVE identifiée, décrivez brièvement le type de vulnérabilité (injection, déni de service, divulgation d'information, etc.). Ces vulnérabilités sont-elles directement exploitables dans votre application ? Justifiez.

1.6 Partie 3 – Pipeline CI/CD avec GitHub Actions

Durée estimée : 90 minutes

1.6.1 Objectifs

- Écrire un pipeline CI/CD complet en 4 jobs enchaînés
- Intégrer Trivy comme gate de sécurité bloquant
- Publier l'image validée sur GitHub Container Registry
- Déployer automatiquement sur Kubernetes

1.6.2 3.1 Structure du pipeline

Votre fichier `.github/workflows/ci-cd.yml` doit implémenter les 4 jobs suivants, dans cet ordre :

```
lint ──▶ scan ──▶ build-push ──▶ deploy
```

Chaque job ne doit s'exécuter **que si le job précédent a réussi**.

1.6.2.1 Job 1 : `lint`

- Vérifie le Dockerfile avec **hadolint**
- Échoue si hadolint détecte des erreurs de niveau `ERROR`
- Référence : https://github.com/hadolint/hadolint

1.6.2.2 Job 2 : `scan`

C'est le job de sécurité central. Il doit :

- Construire l'image Docker **localement sur le runner** sans la pousser vers un registre

- Scanner l'image avec **Trivy**
- **Faire échouer le pipeline** si des vulnérabilités de sévérité `HIGH` ou `CRITICAL` sont trouvées
- Générer un rapport au format **SARIF** et l'uploader dans l'onglet **Security** du dépôt GitHub

  Documentation Trivy : https://trivy.dev/latest/docs/target/container_image/

  💬 **Q3.1** — Quelle est la différence entre `trivy image` et `trivy fs` ? Dans quelle situation utiliseriez-vous l'un plutôt que l'autre ?

  💬 **Q3.2** — Qu'est-ce que le format SARIF ? Quel avantage apporte-t-il par rapport à un rapport texte dans les logs de la CI ?

1.6.2.3 Job 3 : `build-push`

- Ne s'exécute **que si le job `scan` a réussi** (aucune CVE HIGH/CRITICAL)
- Construit et pousse l'image vers `ghcr.io/<votre-username>/taskmanager:<tag>`
- Le tag de l'image doit être basé sur le **SHA du commit** (`github.sha`)
- L'authentification au registre ne doit **pas** contenir de credentials en clair dans le YAML

1.6.2.4 Job 4 : `deploy`

- Ne s'exécute **que sur la branche `main`** (pas sur les branches de feature ni les PRs)
- Applique les manifestes Kubernetes du dossier `k8s/` avec `kubectl`
- Met à jour l'image du Deployment avec le nouveau tag
- Vérifie que le rollout s'est bien terminé avant de conclure le job

  💬 **Q3.3** — Comment stockez-vous vos secrets (kubeconfig, credentials de registre) dans GitHub Actions ? Expliquez le mécanisme utilisé et pourquoi il est plus sûr que de les écrire dans le YAML du pipeline.

1.6.3 3.2 Déclencheurs du pipeline

Configurez le pipeline pour se déclencher :

- Sur chaque `push` vers la branche `main`
- Sur chaque Pull Request ciblant `main`
- Manuellement via `workflow_dispatch`

1.6.4 3.3 Démonstration obligatoire en deux temps

📋 **T3.1 — Scénario de blocage** : poussez le `requirements.txt` original (versions vulnérables) sans le corriger. Le pipeline doit échouer au job `scan`.

Capturez dans votre rendu : - Un screenshot de l'exécution GitHub Actions montrant le job `scan` en rouge - La liste des CVEs affichées dans les logs de ce job

📋 **T3.2 — Scénario de succès** : poussez le `requirements.txt` corrigé. Le pipeline doit enchaîner les 4 jobs et déployer sur Kubernetes.

Capturez dans votre rendu : - Un screenshot de l'exécution GitHub Actions avec les 4 jobs en vert - La sortie `kubectl get pods -n taskmanager` après déploiement réussi

📋 **T3.3** — Naviguez dans l'onglet **Security → Code scanning** de votre dépôt GitHub et copiez un screenshot du rapport SARIF uploadé par Trivy.

1.7 Partie 4 – Manifestes Kubernetes

Durée estimée : 75 minutes

1.7.1 Objectifs

- Écrire tous les manifestes Kubernetes de zéro
- Appliquer les bonnes pratiques de sécurité au niveau des Pods
- Mettre en place un RBAC minimal pour le ServiceAccount du pipeline CI/CD
- Configurer les NetworkPolicies en mode zero-trust

1.7.2 4.1 Ressources à créer

Créez dans le dossier `k8s/` les manifestes pour les ressources suivantes. L'organisation et le nommage des fichiers sont libres, mais doivent rester lisibles et cohérents.

1.7.2.1 Namespace

Créez le namespace `taskmanager` avec les labels appropriés.

1.7.2.2 ResourceQuota et LimitRange

Dans le namespace `taskmanager`, définissez :

- Un `ResourceQuota` limitant l'ensemble du namespace à : **4 CPU**, **1 Gi de mémoire**, **10 Pods maximum**
- Un `LimitRange` imposant des valeurs par défaut de `requests` et `limits` pour tout conteneur qui ne les spécifierait pas explicitement

💬 **Q4.1** — Quelle est la différence fonctionnelle entre un `ResourceQuota` et un `LimitRange` ? Pourquoi est-il recommandé de déployer les deux ensemble ?

📋 **T4.1** — Après avoir appliqué vos manifestes, copiez dans votre rendu la sortie de :

```
kubectl describe resourcequota -n taskmanager
kubectl describe limitrange -n taskmanager
```

1.7.2.3 Redis

Déployez Redis (`redis:7-alpine`) dans le namespace `taskmanager` avec :

- Un mot de passe stocké dans un `Secret` Kubernetes (jamais en clair dans le manifeste)
- Des sondes `liveness` et `readiness` adaptées
- Des `requests` et `limits` de ressources définies
- Un `PersistentVolumeClaim` pour la persistance des données

1.7.2.4 Déploiement de l'application

Créez un `Deployment` pour `taskmanager` respectant les contraintes suivantes :

Contrainte	Valeur attendue
Replicas	2 minimum
Image	`ghcr.io/<votre-username>/taskmanager:<sha>`
`runAsNonRoot`	`true`
`readOnlyRootFilesystem`	`true`
`allowPrivilegeEscalation`	`false`
Capabilities Linux	Toutes supprimées (`drop: ["ALL"]`)
Variables d'env sensibles	Injectées depuis un `Secret`, jamais en clair
Sonde liveness	HTTP GET `/healthz` sur le port 5000
Sonde readiness	HTTP GET `/readyz` sur le port 5000
Stratégie de mise à jour	RollingUpdate, `maxUnavailable: 0`, `maxSurge: 1`

> Si `readOnlyRootFilesystem: true` provoque des erreurs au démarrage de gunicorn, identifiez les chemins qui nécessitent de l'écriture et montez des volumes `emptyDir` uniquement pour ces chemins.

> 💬 **Q4.2** — Expliquez le rôle de chacune des contraintes `securityContext` listées ci-dessus. Pourquoi `drop: ["ALL"]` est-il une bonne pratique, et quelles capabilities re-ajouteriez-vous si l'application en avait besoin ?

> 📋 **T4.2** — Vérifiez en pratique que votre Deployment respecte bien les contraintes de sécurité. Copiez dans votre rendu la sortie de :

```
kubectl get deployment taskmanager -n taskmanager -o
        jsonpath='{.spec.template.spec.containers[0].securityContext}' | python3 -
        m json.tool
```

1.7.2.5 Services

Créez les Services nécessaires pour exposer l'application. Justifiez dans vos réponses le type de Service choisi pour chaque composant (`ClusterIP`, `NodePort`, ou `LoadBalancer`).

> 📋 **T4.3** — Testez l'accès à l'application via son Service NodePort. Copiez dans votre rendu le résultat de l'appel à `/healthz` et `/tasks`, ainsi que la commande utilisée.

1.7.2.6 Ingress

Exposez l'application via un Ingress sur le hostname **`taskmanager.local`**.

> 💬 **Q4.3** — Sur minikube, quelle manipulation est nécessaire pour accéder à `taskmanager.local` depuis un navigateur ? Pourquoi cette étape est-elle nécessaire (que se passe-t-il au niveau réseau) ?

> 📋 **T4.4** — Accédez à l'interface web `http://taskmanager.local` dans votre navigateur. Capturez un screenshot de l'interface avec au moins une tâche créée.

1.7.2.7 HorizontalPodAutoscaler

Configurez un HPA sur le Deployment `taskmanager` :

- Minimum : **2 replicas** — Maximum : **6 replicas**
- Déclenchement sur l'utilisation CPU (seuil à définir et justifier)

📋 **T4.5** — Simulez une montée en charge pour déclencher le HPA. Lancez un Pod de génération de charge dans le namespace `taskmanager`, puis observez le comportement du HPA :

```
# Point de départ — adaptez la cible et la durée
kubectl run load-generator --image=busybox --restart=Never -n taskmanager \
  -- /bin/sh -c "while true; do wget -q -O- http://taskmanager/tasks; done"
```

Copiez dans votre rendu la sortie de `kubectl get hpa -n taskmanager` avant, pendant et après la charge. Supprimez ensuite le Pod `load-generator`.

1.7.3 4.2 RBAC pour le pipeline CI/CD

Le job `deploy` de votre pipeline a besoin d'interagir avec l'API Kubernetes pour mettre à jour les Deployments. Ce job ne doit **pas** utiliser un kubeconfig administrateur avec des droits illimités.

Créez :

- Un `ServiceAccount` nommé `github-actions` dans le namespace `taskmanager`
- Un `Role` accordant **uniquement** les permissions strictement nécessaires aux opérations de déploiement
- Un `RoleBinding` associant les deux

💬 **Q4.4** — Listez précisément les verbes (`verbs`) et les ressources (`resources`) que vous avez accordés au `ServiceAccount` `github-actions`. Justifiez chaque permission. Quelles permissions avez-vous délibérément refusées ?

💬 **Q4.5** — Quelle est la différence entre `Role`/`RoleBinding` et `ClusterRole`/`ClusterRoleBinding` ? Pourquoi avez-vous utilisé l'un plutôt que l'autre ici ?

📋 **T4.6** — Vérifiez en pratique les permissions accordées au ServiceAccount. Utilisez `kubectl auth can-i` avec l'impersonification pour tester ce que le ServiceAccount peut et ne peut pas faire :

```
# Exemple de point de départ — testez plusieurs actions
kubectl auth can-i get deployments \
  --as=system:serviceaccount:taskmanager:github-actions \
  -n taskmanager

kubectl auth can-i delete namespaces \
  --as=system:serviceaccount:taskmanager:github-actions
```

Copiez dans votre rendu au moins **5 tests** différents (actions autorisées et refusées) avec leurs résultats.

1.7.4 4.3 NetworkPolicy

Implémentez une politique réseau **zero-trust** pour le namespace `taskmanager`.

Avant d'écrire le moindre YAML, réalisez le schéma des flux réseau légitimes entre les composants (sens, port, protocole). Incluez ce schéma dans votre rendu.

Vos NetworkPolicies doivent :

- Bloquer **tout** le trafic par défaut (ingress et egress) dans le namespace
- Autoriser uniquement les flux identifiés dans votre schéma
- Autoriser la résolution DNS (indispensable au fonctionnement des Pods)

💬 **Q4.6** — Expliquez pourquoi une règle "default deny all" est considérée comme la bonne pratique en matière de sécurité réseau Kubernetes. Quels sont les risques concrets si on ne la met pas en place ?

📋 **T4.7** — Vérifiez que vos NetworkPolicies fonctionnent correctement en testant des flux autorisés et des flux bloqués depuis un Pod de debug. Voici un point de départ :

```
# Lancez un Pod de debug dans le namespace
kubectl run debug --image=busybox --restart=Never -n taskmanager -- sleep 3600

# Testez les connexions (adaptez les destinations et ports)
kubectl exec debug -n taskmanager -- wget -qO- --timeout=3
        http://taskmanager/healthz
kubectl exec debug -n taskmanager -- wget -qO- --timeout=3 http://redis:6379
```

Copiez dans votre rendu le résultat d'au moins **3 tests** (flux attendus autorisés et bloqués). Supprimez ensuite le Pod `debug`.

1.8 Partie 5 – Sécurité avancée avec Trivy Operator

Durée estimée : 45 minutes

1.8.1 Objectifs

- Installer Trivy Operator dans le cluster Kubernetes
- Comprendre la complémentarité entre scan CI et scan in-cluster
- Analyser et interpréter les rapports de vulnérabilités générés automatiquement

1.8.2 5.1 Installation de Trivy Operator

Trivy Operator est un contrôleur Kubernetes qui surveille **en continu** les workloads et génère des rapports de sécurité sous forme de CRDs.

📋 **T5.1** — Installez Trivy Operator via Helm dans un namespace `trivy-system`. Copiez dans votre rendu :

- La commande Helm utilisée
- La sortie de `kubectl get pods -n trivy-system`
- La sortie de `kubectl get crd | grep trivy`

💬 **Q5.1** — Listez les CRDs créées par Trivy Operator. Quelle est la différence entre un `VulnerabilityReport` et un `ConfigAuditReport` ?

1.8.3 5.2 Analyse des rapports de vulnérabilités

📋 **T5.2** — Une fois les scans terminés (attendez 2-3 minutes), listez et décrivez les rapports de votre Deployment :

```
kubectl get vulnerabilityreports -n taskmanager
kubectl describe vulnerabilityreport <nom-du-rapport> -n taskmanager
```

Copiez dans votre rendu le résumé des vulnérabilités trouvées (total par sévérité).

💬 **Q5.2** — Le nombre de vulnérabilités rapportées par Trivy Operator est-il cohérent avec le résultat de votre scan Trivy en CI ? Expliquez les éventuelles différences.

1.8.4 5.3 Analyse des rapports de configuration

📋 **T5.3** — Consultez le `ConfigAuditReport` de votre Deployment et copiez dans votre rendu les 3 violations les plus critiques détectées. Pour chacune, expliquez ce qu'elle signifie et comment la corriger dans votre manifeste.

💬 **Q5.3** — Parmi les violations identifiées, lesquelles avez-vous déjà corrigées dans votre Deployment (securityContext, etc.) ? Laquelle vous a semblé la plus difficile à corriger et pourquoi ?

1.8.5 5.4 Comparaison des deux approches

💬 **Q5.4** — Rédigez un paragraphe comparant les deux approches de scanning (10 à 15 lignes minimum) :

- **Shift-left** : scan dans le pipeline CI avant tout déploiement (Trivy dans GitHub Actions)
- **Runtime scanning** : surveillance continue dans le cluster (Trivy Operator)

Quand faut-il utiliser l'une, l'autre, ou les deux simultanément ? Quelles sont les limites de chaque approche ?

1.9 Partie 6 – Questions de synthèse

Ces questions sont à inclure dans votre document de rendu.

1.9.1 6.1 Chaîne de sécurité complète

💬 **Q6.1** — Décrivez ou dessinez la chaîne de sécurité complète mise en place dans ce TP, depuis le `git push` du développeur jusqu'au Pod en cours d'exécution dans Kubernetes. Identifiez et nommez les **5 points de contrôle de sécurité** clés et expliquez ce que chacun protège.

1.9.2 6.2 Gestion d'une vulnérabilité sans correctif

💬 **Q6.2** — Une CVE `CRITICAL` est découverte par Trivy dans une dépendance Python de votre application, mais aucune version corrigée n'est disponible. Décrivez la procédure que vous suivriez pour :

1. Évaluer si la CVE est réellement exploitable dans votre contexte applicatif
2. Décider si le pipeline doit continuer à bloquer ou si une exception doit être accordée

3. Documenter et suivre cette exception de façon traçable (indice : fichier `.trivyignore`)

💬 **Q6.3** — Quelle est la différence entre une **CVE** et un **faux positif** dans le contexte de Trivy ? Donnez un exemple concret de faux positif et expliquez comment le gérer.

1.9.3 6.3 GitOps et amélioration continue

💬 **Q6.4** — Le pattern **GitOps** (Flux CD, ArgoCD) est souvent présenté comme une alternative plus robuste à `kubectl apply` dans un pipeline CI/CD. Expliquez le principe du GitOps et en quoi il améliore la traçabilité et la sécurité des déploiements par rapport à l'approche utilisée dans ce TP.

💬 **Q6.5** — Si vous deviez préparer ce pipeline pour un environnement de production réel, quels seraient les **3 axes d'amélioration les plus importants** à mettre en place ? Justifiez chaque choix.

1.9.4 6.4 Retour d'expérience

💬 **Q6.6** — Décrivez les **deux problèmes techniques les plus significatifs** que vous avez rencontrés durant ce TP et la démarche que vous avez suivie pour les résoudre. *(Réponse libre — pas de bonne ou mauvaise réponse.)*

1.10 Livrables et évaluation

1.10.1 Ce que vous devez rendre

Livrable	Description
URL du dépôt GitHub	Dépôt **public** avec l'ensemble du code
Captures T3.1 et T3.2	Pipeline échoué (CVEs) + pipeline réussi (4 jobs au vert)
Captures T4.4	Interface web accessible via `taskmanager.local`
Capture T3.3	Rapport SARIF dans l'onglet Security du dépôt
Sorties des tâches pratiques	Toutes les sorties de commandes demandées (T1.3, T2.2, T4.1, T4.2, T4.6, T4.7, T5.1, T5.2)
Dossier `k8s/` complet	Tous les manifestes Kubernetes, organisés et commentés
Réponses aux questions	Toutes les réponses Q, dans le même document

Un pipeline fonctionnel à 80% mais clairement documenté et justifié vaut mieux qu'un pipeline à 100% sans compréhension des choix effectués.

1.10.2 Critères de qualité transversaux

- **Sécurité** : aucune credential en clair dans le code, les manifestes ou les logs de pipeline
- **Lisibilité** : manifestes organisés, commentés, nommage cohérent
- **Principe du moindre privilège** : RBAC accordant exactement ce qui est nécessaire
- **Fiabilité** : sondes correctement configurées, rolling update sans interruption de service

---

1.11 Ressources

1.11.1 Documentation officielle

Ressource	URL
Trivy – Scanner d'images	https://trivy.dev/latest/docs/target/container_image/
Trivy – Format SARIF	https://trivy.dev/latest/docs/configuration/reporting/
Trivy – Ignorer des CVEs	https://trivy.dev/latest/docs/configuration/filtering/
Trivy Operator	https://trivy.dev/latest/docs/target/kubernetes/
GitHub Actions – Workflows	https://docs.github.com/en/actions/writing-workflows
GitHub Actions – Secrets	https://docs.github.com/en/actions/security-for-github-actions/security-guides/using-secrets-in-github-actions
GitHub Container Registry	https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry
Kubernetes – Security Context	https://kubernetes.io/docs/tasks/configure-pod-container/security-context/
Kubernetes – RBAC	https://kubernetes.io/docs/reference/access-authn-authz/rbac/
Kubernetes – NetworkPolicy	https://kubernetes.io/docs/concepts/services-networking/network-policies/
Kubernetes – HPA	https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/
hadolint	https://github.com/hadolint/hadolint
Helm – Installation	https://helm.sh/docs/intro/install/

1.11.2 Commandes de départ (non exhaustif)

Ces commandes sont des points de départ — les flags et options sont à adapter à votre contexte :

```
# Vérifier les outils installés
minikube version && kubectl version --client && trivy --version && helm version

# Démarrer minikube avec les addons nécessaires
minikube start --cpus=2 --memory=3072 --driver=docker
minikube addons enable ingress
minikube addons enable metrics-server

# Scanner une image localement avec Trivy
trivy image --severity HIGH,CRITICAL <image:tag>

# Générer un rapport SARIF
trivy image --format sarif --output trivy-results.sarif <image:tag>

# Tester les permissions d'un ServiceAccount
kubectl auth can-i <verbe> <ressource> \
  --as=system:serviceaccount:<namespace>:<nom-du-sa> -n <namespace>

# Ajouter le repo Helm de Trivy Operator
helm repo add aqua https://aquasecurity.github.io/helm-charts/
helm repo update

# Lister les rapports Trivy Operator
```

```
kubectl get vulnerabilityreports -A
kubectl get configauditreports -A

# Accéder à l'application (macOS/Linux avec minikube)
minikube tunnel
# Puis ajouter dans /etc/hosts : <IP_MINIKUBE> taskmanager.local
```

1.11.3 Conseils méthodologiques

- **Testez chaque élément localement** avant de l'intégrer dans le pipeline
- **Commitez fréquemment** avec des messages clairs — votre historique git fait partie de l'évaluation
- **Lisez les messages d'erreur en entier** — Kubernetes et GitHub Actions sont verbeux et les messages sont généralement explicites
- **Avancez par parties successives** — ne bloquez pas sur une partie, passez à la suivante et revenez
- **La documentation officielle est votre meilleure ressource** — préférez-la aux forums et blogs

---

Bonne chance. L'objectif n'est pas la perfection, c'est la compréhension.